

Quaternions

Quaternions can be described as a 4 dimensional number system which can be used for rotations.

$$i^2 = j^2 = k^2 = ijk = 1$$

A quaternion can be described as: $q = w + i + j + k$

Product of Imaginary numbers:

Product	i	j	K
K	-1	k	-j
J	-k	-1	i
I	j	-i	-1

Quaternion algebra:

Quaternions can be split into 2 parts a vector part $v = (x, y, z)$ and a scalar part ($s = w$)

$q = (s, v)$, with $s = w$ real and $v = (x, y, z)$ imaginary. A quaternion where $S = 0$ is called a pure quaternions and a quaternion with $V = 0$ is just a real number.

- The conjugate: $q^* = (s, -v)$
- Norm: $|q|^2 = |qq|$
- Inverse: $q^{-1} = \frac{q^*}{|q|^2}$

Quaternion algebra:

Quaternions can be split into 2 parts a vector part $v = (x, y, z)$ and a scalar part ($s = w$)

$q = (s, v)$, with $s = w$ real and $v = (x, y, z)$ imaginary. A quaternion where $S = 0$ is called a pure quaternions and a quaternion with $V = 0$ is just a real number.

- The conjugate: $q^* = (s, -v)$
- Norm: $|q|^2 = |qq|$
- Inverse: $q^{-1} = \frac{q^*}{|q|^2}$

Multiplying 2 Quaternions:

$$q_1 q_2 = (w_1 + x_1 i + y_1 j + z_1 k)(w_2 + x_2 i + y_2 j + z_2 k)$$

$$w_1(w_2 + x_2 i + y_2 j + z_2 k) = w_1 w_2 + w_1 x_2 i + w_1 y_2 j + w_1 z_2 k$$

$$x_1 i(w_2 + x_2 i + y_2 j + z_2 k) = x_1 w_2 i + x_1 x_2 i^2 + x_1 y_2 ij + x_1 z_2 ik = x_1 w_2 i - x_1 x_2 + x_1 y_2 k - x_1 z_2 j$$

$$y_1 j(w_2 + x_2 i + y_2 j + z_2 k) = y_1 w_2 j + y_1 x_2 ji + y_1 y_2 j^2 + y_1 z_2 jk = y_1 w_2 j - y_1 x_2 k - y_1 y_2 + y_1 z_2 i$$

$$z_1 k(w_2 + x_2 i + y_2 j + z_2 k) = z_1 w_2 k + z_1 x_2 ki + z_1 y_2 kj + z_1 z_2 k^2 = z_1 w_2 k + z_1 x_2 j - z_1 y_2 i - z_1 z_2$$

Collect the terms for (w,i,j,k)

$$w(w_1 w_2 - x_1 x_2 - y_1 y_2 - z_1 z_2)$$

$$i(w_1 x_2 + x_1 w_2 + y_1 z_2 - z_1 x_2)$$

$$j(w_1 y_2 - x_1 z_2 + y_1 w_2 + z_1 x_2)$$

$$k(w_1 z_2 + x_1 y_2 - y_1 x_2 + z_1 w_2)$$

$$q_1 q_2 = (w_1 w_2 - x_1 x_2 - y_1 y_2 - z_1 z_2) + i(w_1 x_2 + x_1 w_2 + y_1 z_2 - z_1 x_2) + j(w_1 y_2 - x_1 z_2 + y_1 w_2 + z_1 x_2) + k(w_1 z_2 + x_1 y_2 - y_1 x_2 + z_1 w_2)$$

Multiplying Quaternion by Pure quaternion

Pure(q) = W

$$qw = (w + xi + yj + zk)(\omega_x i + \omega_y j + \omega_z k)$$

$$w(\omega_x i + \omega_y j + \omega_z k) = w\omega_x i + w\omega_y j + w\omega_z k$$

$$xi(\omega_x i + \omega_y j + \omega_z k) = x\omega_x i^2 + x\omega_y ij + x\omega_z ik = -x\omega_x + x\omega_y k - x\omega_z j$$

$$yj(\omega_x i + \omega_y j + \omega_z k) = y\omega_x ji + y\omega_y j^2 + y\omega_z jk = -y\omega_x k - y\omega_y + y\omega_z i$$

$$zk(\omega_x i + \omega_y j + \omega_z k) = z\omega_x ki + z\omega_y kj + z\omega_z k^2 = z\omega_x j - z\omega_y i - z\omega_z$$

Collect the terms for (w,i,j,k)

$$-x\omega_x - y\omega_y - z\omega_z = (-x\omega_x - y\omega_y - z\omega_z)1$$

$$(w\omega_x i + y\omega_z i - z\omega_y i) = (w\omega_x + y\omega_z - z\omega_y)i$$

$$(w\omega_y j - x\omega_z j + z\omega_x j) = (w\omega_y - x\omega_z + z\omega_x)j$$

$$w\omega_z k + x\omega_y k - y\omega_x k = (w\omega_z + x\omega_y - y\omega_x)k$$

$$qw = (-x\omega_x - y\omega_y - z\omega_z) + (w\omega_x + y\omega_z - z\omega_y)i + (w\omega_y - x\omega_z + z\omega_x)j + (w\omega_z + x\omega_y - y\omega_x)k$$

Rotating vector using Quaternion

For rotation a pure quaternion is used to store the (x,y,z) vector in (i,j,k) axis, there's a set of very basic steps used to rotate a quaternion. Firstly the axis system is defined as (x,y,z | i,j,k) where x is roll, y is pitch and z is yaw these are all taken from body axis. The basic step to rotating a quaternion is to have a quaternion with a real part and a imaginary part to be rotated about for example rolling around x axis consist of using $q = \cos + i \sin$ but this is only per axis the formal description is:

$$q = \cos\left(\frac{\theta}{2}\right) + \sin\left(\frac{\theta}{2}\right)(n_x i + n_y j + n_z k)$$

Term: $(n_x i + n_y j + n_z k)$

$(\sqrt{i^2 + j^2 + k^2} = 1)$ for $(n_x i + n_y j + n_z k)$ this specifically describes a vector pointing in a direction which rotates in the angle of θ . So say the vector points (1,0,0) $q = \cos\left(\frac{\theta}{2}\right) + \sin\left(\frac{\theta}{2}\right)(1i + 0j + 0k) = \cos\left(\frac{\theta}{2}\right) + \sin\left(\frac{\theta}{2}\right)i$ this defines the rotation around the body roll axis, the body orientation is defined by a pure quaternion $V = (i,j,k)$ where q isn't pure due to the cos term being real. Taking the quaternion q and multiplying by V allows the vector to be rotated about a axis. This is easy to keep track when applying a single rotation, meaning performing a rotation of roll, pitch or yaw at a time only require to:

Motion axis	q	q*
Roll	$q_x = \cos\left(\frac{\theta}{2}\right) + \sin\left(\frac{\theta}{2}\right)i$	$q_x^* = \cos\left(\frac{\theta}{2}\right) - \sin\left(\frac{\theta}{2}\right)i$
Pitch	$q_y = \cos\left(\frac{\theta}{2}\right) + \sin\left(\frac{\theta}{2}\right>j$	$q_y^* = \cos\left(\frac{\theta}{2}\right) - \sin\left(\frac{\theta}{2}\right>j$
Yaw	$q_z = \cos\left(\frac{\theta}{2}\right) + \sin\left(\frac{\theta}{2}\right>k$	$q_z^* = \cos\left(\frac{\theta}{2}\right) - \sin\left(\frac{\theta}{2}\right>k$

Example 1: given a Pure quaternion has vector $v = (1,0,0)$ to be rotated 90 degrees in the pitch axis j

$$\left(q_y = \cos\left(\frac{90}{2}\right) + \sin\left(\frac{90}{2}\right>j = \frac{\sqrt{2}}{2} + \frac{\sqrt{2}}{2}j\right) \quad (v = i + 0j + 0k = i) \quad (q_y^* = \cos\left(\frac{90}{2}\right) - \sin\left(\frac{90}{2}\right>j = \frac{\sqrt{2}}{2} - \frac{\sqrt{2}}{2}j)$$

$$q_y v = \left(\frac{\sqrt{2}}{2} + \frac{\sqrt{2}}{2}j\right)i = \frac{\sqrt{2}}{2}i + \frac{\sqrt{2}}{2}ji = \frac{\sqrt{2}}{2}i - \frac{\sqrt{2}}{2}k$$

$$q_y v q_y^* = \left(\frac{\sqrt{2}}{2}i - \frac{\sqrt{2}}{2}k\right)\left(\frac{\sqrt{2}}{2} - \frac{\sqrt{2}}{2}j\right) = \frac{1}{2}i - \frac{1}{2}ij - \frac{1}{2}k - \frac{1}{2}kj = \frac{1}{2}i - \frac{1}{2}k - \frac{1}{2}k - \frac{1}{2}i = -k$$

Example 2: the previous vector was rotated around $v = (1,0,0)$ to $v_2 = (0,0,-1)$ now roll the vector by 90 degrees using roll

$$\left(q_x = \cos\left(\frac{90}{2}\right) + \sin\left(\frac{90}{2}\right>i = \frac{\sqrt{2}}{2} + \frac{\sqrt{2}}{2}i\right) \quad (v = 0i + 0j - k = -k) \quad (q_x^* = \cos\left(\frac{90}{2}\right) - \sin\left(\frac{90}{2}\right>i = \frac{\sqrt{2}}{2} - \frac{\sqrt{2}}{2}i)$$

$$q_x v = \left(\frac{\sqrt{2}}{2} + \frac{\sqrt{2}}{2}i\right)(-k) = -\frac{\sqrt{2}}{2}k - \frac{\sqrt{2}}{2}ik = -\frac{\sqrt{2}}{2}k + \frac{\sqrt{2}}{2}j$$

$$q_x v q_x^* = \left(-\frac{\sqrt{2}}{2}k + \frac{\sqrt{2}}{2}j\right)\left(\frac{\sqrt{2}}{2} - \frac{\sqrt{2}}{2}i\right) = -\frac{\sqrt{2}}{2}k + \frac{1}{2}j + \frac{1}{2}j + \frac{1}{2}k = j$$

Example 3: take $v = (1,0,0)$ rotate it around yaw axis at 90 degrees around the Z axis

$$\left(q_z = \cos\left(\frac{90}{2}\right) + \sin\left(\frac{90}{2}\right>k = \frac{\sqrt{2}}{2} + \frac{\sqrt{2}}{2}k\right) \quad (v = i + 0j + 0k = i) \quad (q_z^* = \cos\left(\frac{90}{2}\right) - \sin\left(\frac{90}{2}\right>k = \frac{\sqrt{2}}{2} - \frac{\sqrt{2}}{2}k)$$

$$q_z v = \left(\frac{\sqrt{2}}{2} + \frac{\sqrt{2}}{2}k\right)i = \frac{\sqrt{2}}{2}i + \frac{\sqrt{2}}{2}ik = \frac{\sqrt{2}}{2}i + \frac{\sqrt{2}}{2}j$$

$$q_z v q_z^* = \left(\frac{\sqrt{2}}{2}i + \frac{\sqrt{2}}{2}j\right)\left(\frac{\sqrt{2}}{2} - \frac{\sqrt{2}}{2}k\right) = \frac{1}{2}i + \frac{1}{2}j + \frac{1}{2}j - \frac{1}{2}i = j$$

Observations of examples

Taking example 1,2,3 there's a few complications which arise, mainly being that the purpose of quaternions in rotation setting is that it can be used to rotate a body and be not be prone to gimbal lock, this means that the quaternion doesn't loose track of the axis since the result of example 2 and example 3 are the same yet should have different orientation shows a very important aspect on how the quaternions need to be setup for rotation this being the body and world axis system.

Rotating a body axis around world axis using Quaternions:

Since quaternion can be defined as a rotation it can also be used to define body orientation around a world axis, this is mainly what allows the Quaternion to be prone of gimbal lock its commutative meaning will translates what the next rotation will be based on the last this is a commutativity between the world and body. It can be set up as so:

Define the Quaternion:	Define the body axis:
Unit Quaternion: $Q = w + xi + yj + zk$	Body x (nose/roll axis) $\rightarrow i = (1,0,0)$
Start identity: $Q = w + 0i + 0j + 0k$	Body y (right/pitch axis) $\rightarrow j = (0,1,0)$
	Body z (up/Yaw axis) $\rightarrow k = (0,0,1)$

Example : pitch 90 degrees then roll 90 degrees

$$(q_y = \cos\left(\frac{90}{2}\right) + \sin\left(\frac{90}{2}\right)j = \frac{\sqrt{2}}{2} + \frac{\sqrt{2}}{2}j) \quad (v = i + 0j + 0k = i) \quad (q_y^* = \cos\left(\frac{90}{2}\right) - \sin\left(\frac{90}{2}\right)j = \frac{\sqrt{2}}{2} - \frac{\sqrt{2}}{2}j)$$

$$q_y i = \left(\frac{\sqrt{2}}{2} + \frac{\sqrt{2}}{2}j\right)i = \frac{\sqrt{2}}{2}i + \frac{\sqrt{2}}{2}ji = \frac{\sqrt{2}}{2}i - \frac{\sqrt{2}}{2}k$$

$$q_y i q_y^* = \left(\frac{\sqrt{2}}{2}i - \frac{\sqrt{2}}{2}k\right)\left(\frac{\sqrt{2}}{2} - \frac{\sqrt{2}}{2}j\right) = \frac{1}{2}i - \frac{1}{2}k - \frac{1}{2}k - \frac{1}{2}i = -k$$

$$q_y j = \left(\frac{\sqrt{2}}{2} + \frac{\sqrt{2}}{2}j\right)j = \frac{\sqrt{2}}{2}j + \frac{\sqrt{2}}{2}j^2 = \frac{\sqrt{2}}{2}j - \frac{\sqrt{2}}{2}$$

$$q_y j q_y^* = \left(\frac{\sqrt{2}}{2}j - \frac{\sqrt{2}}{2}\right)\left(\frac{\sqrt{2}}{2} - \frac{\sqrt{2}}{2}j\right) = -\frac{1}{2} + \frac{1}{2}j + \frac{1}{2}j + \frac{1}{2} = j$$

$$q_y k = \left(\frac{\sqrt{2}}{2} + \frac{\sqrt{2}}{2}j\right)k = \frac{\sqrt{2}}{2}k + \frac{\sqrt{2}}{2}jk = \frac{\sqrt{2}}{2}k + \frac{\sqrt{2}}{2}i$$

$$q_y k = \left(\frac{\sqrt{2}}{2}i + \frac{\sqrt{2}}{2}k\right)\left(\frac{\sqrt{2}}{2} - \frac{\sqrt{2}}{2}j\right) = \frac{1}{2}i - \frac{1}{2}k + \frac{1}{2}k + \frac{1}{2}i = i$$

This is a full pitch rotation of 90 degrees, now the body axis is defined by { nose = (0,0,-1), bottom (0,1,0), side tip (1,0,0) which makes this makes sense and keeps the orientation to a reference axis now to apply roll multiply it again by the roll quaternion but now with the new axis.

$$(q_x = \cos\left(\frac{90}{2}\right) + \sin\left(\frac{90}{2}\right)k = \frac{\sqrt{2}}{2} + \frac{\sqrt{2}}{2}k) \quad (v = 0i + 0j - k = -k) \quad (q_x^* = \cos\left(\frac{90}{2}\right) - \sin\left(\frac{90}{2}\right)k = \frac{\sqrt{2}}{2} - \frac{\sqrt{2}}{2}k)$$

$$[nose] q_x (-k) q_x^* = \left(\frac{\sqrt{2}}{2} - \frac{\sqrt{2}}{2}k\right)(-k)\left(\frac{\sqrt{2}}{2} + \frac{\sqrt{2}}{2}k\right) = \left(-\frac{\sqrt{2}}{2}k - \frac{\sqrt{2}}{2}\right)\left(\frac{\sqrt{2}}{2} + \frac{\sqrt{2}}{2}k\right) = -\frac{1}{2}k + \frac{1}{2} - \frac{1}{2} - \frac{1}{2}k = -k$$

$$[wing tip] q_x j q_x^* = \left(\frac{\sqrt{2}}{2} - \frac{\sqrt{2}}{2}k\right)j\left(\frac{\sqrt{2}}{2} + \frac{\sqrt{2}}{2}k\right) = \left(\frac{\sqrt{2}}{2}j + \frac{\sqrt{2}}{2}i\right)\left(\frac{\sqrt{2}}{2} + \frac{\sqrt{2}}{2}k\right) = \frac{1}{2}j + \frac{1}{2}i + \frac{1}{2}i - \frac{1}{2}j = i$$

$$[top] q_x i q_x^* = \left(\frac{\sqrt{2}}{2} - \frac{\sqrt{2}}{2}k\right)i\left(\frac{\sqrt{2}}{2} + \frac{\sqrt{2}}{2}k\right) = \left(\frac{\sqrt{2}}{2}i - \frac{\sqrt{2}}{2}j\right)\left(\frac{\sqrt{2}}{2} + \frac{\sqrt{2}}{2}k\right) = \frac{1}{2}i - \frac{1}{2}j - \frac{1}{2}j - \frac{1}{2}i = -j$$

From this example there's a track of orientation per rotation initially it starts as 3 points [vi(1,0,0), vj(0,1,0), vz(0,0,1)] [vi=nose, vj = left wing, vz= top] so when all are 1 then they're just vectors perpendicular to each other the first step was to pitch 90 degrees (positive angle pitches down) this leaves vector vi = (0,0,-1) meaning nose pointing directly down, vj = (0,1,0) this direction doesn't change since it's the axis being rotated about and vz = (1,0,0) where now the top points where the nose was previously pointing which is the correct output for the given rotation. The second step is to roll the new orientation which is rolling a aircraft pointing up [vi(0,0,-1), vj(0,1,0), vz(1,0,0)] so when rolling another 90 degrees and applying the same pattern it yields [vi(0,0,-1), vj(1,0,0), vz(0,-1,0)] its all very logical but computationally expensive it serves well as a example but for application best to use quaternion composition.

Quaternion composition

Given a quaternion $Q = 1$, can be modelled as 3 vectors already $[v_i(1,0,0), v_j(0,1,0), v_z(0,0,1)]$ this can be used to describe orientation and keep its state without needing to decompose the vectors each time. Initially when $Q = 1$ world and body axis are aligned and rotating the body axis around the world axis the quaternions only needs to be multiplied by a rotation quaternion. This is the correct way of using a quaternion since it's the least computationally expensive method of rotating any body in a world frame.

Example: pitch 90 degrees then roll 90 degrees:

$$Q_0 = 1 + 0i + 0j + 0k$$

$$q_y = \cos\left(\frac{90}{2}\right) + \sin\left(\frac{90}{2}\right)j = \frac{\sqrt{2}}{2} + \frac{\sqrt{2}}{2}j, q_x = \cos\left(\frac{90}{2}\right) + \sin\left(\frac{90}{2}\right)i = \frac{\sqrt{2}}{2} + \frac{\sqrt{2}}{2}i$$

$$Q_1 = Q q_y = (1 + 0i + 0j + 0k)\left(\frac{\sqrt{2}}{2} + \frac{\sqrt{2}}{2}j\right) = \frac{\sqrt{2}}{2} + \frac{\sqrt{2}}{2}j$$

$$Q_2 = Q_1 q_x = \left(\frac{\sqrt{2}}{2} + \frac{\sqrt{2}}{2}j\right)\left(\frac{\sqrt{2}}{2} + \frac{\sqrt{2}}{2}i\right) = \frac{1}{2}(1 + i + j - k)$$

$$Q_2^* = \frac{1}{2}(1 - i - j + k)$$

The Quaternion Q_2 holds orientation of body relative to the world axis, this form is less intuitive but more computationally efficient since using this allows the full orientation to be stored in 4 numbers which can be used to extract 3 vectors which can be used to deduct a rotation matrix or Euler angles. To extract these vectors the quaternion must be multiplied by the body axis and the conjugate. Here I will decompose Q_2 .

$$Q_2 i Q_2^* = \frac{1}{2}(1 + i + j - k)i\frac{1}{2}(1 + i + j - k) = -k$$

$$Q_2 j Q_2^* = \frac{1}{2}(1 + i + j - k)j\frac{1}{2}(1 + i + j - k) = i$$

$$Q_2 k Q_2^* = \frac{1}{2}(1 + i + j - k)k\frac{1}{2}(1 + i + j - k) = -j$$

This shows 2 rotations applied and decomposed, compared to previous steps it can be seen that there's much less computation needed to achieve the same operation which is the reason for why most computed graphics involving rotation or flight instruments rely on Quaternions its much less prone to process noise.

Key takeaway:

- Rotation Quaternion q_x, q_y, q_z
- Orientation Quaternion Q
- Decomposition:
 - Nose $Q_i Q^*$
 - Left tip $Q_j Q^*$
 - Top $Q_k Q^*$

Practical architectural insight (for programs)

The decomposition is by far the most complex part of this system, to keep the system sane its extremely important to run the program in a manner which the least amount of information is lost. For a single core MCU the loop can run on quaternion and through a trigger decomposing the quaternion to be used say for displaying, main issue is when the loop updates output decomposed vector a few cycles of rotation may be lost, if doing on a dual core MCU its more effective since a single core can be used to continuously to integrate rotations and another core for decomposing. Now to use quaternions in a program using the same method as before is still much to computationally expensive since it involves the use of trig functions which digital systems tend to not like, so using a discrete integration method with rads/s as input is optimal instead of continuous values.

Quaternion integration

Integrating Quaternion is the final point in terms of using Quaternions in a INS/IMU system, gyroscopes have a input of rads/s not definite angles. Say you take the gyro and integrate rads/s into degrees then to quaternion this is very heavy computationally so instead taking the rads/s and integrating quaternions is optimal.

Looking at how to represent quaternions as rad/s for small angles:

$$\theta_x = \omega_x dt, \quad \theta_y = \omega_y dt, \quad \theta_z = \omega_z dt$$

$$\cos\left(\frac{\theta}{2}\right) \approx 1, \quad \sin\left(\frac{\theta}{2}\right) \approx \frac{\theta}{2}$$

$$q_x \approx 1 + \frac{\omega_x dt}{2} i, \quad q_y \approx 1 + \frac{\omega_y dt}{2} j, \quad q_z \approx 1 + \frac{\omega_z dt}{2} k$$

Applying to orientation Q:

$$Q_{new} = Q q_{step} = Q \left(1 + \frac{dt}{2} (\omega_x i + \omega_y j + \omega_z k) \right) = Q + Q \frac{dt}{2} (\omega_x i + \omega_y j + \omega_z k)$$

$$\frac{dQ}{dt} = \frac{Q_{new} - Q}{dt}$$

$$\frac{dQ}{dt} = \frac{1}{2} Q \omega_q = \frac{1}{2} Q (\omega_x i + \omega_y j + \omega_z k)$$

This is mainly just multiplying a Quaternion by a pure Quaternion, the pure quaternion per cycle holds rads/s for each axis its multiplied by the previous quaternion * 0.5 to update the rotation velocity and direction with respect to world axis, a general expansion of multiplying a quaternion by a pure quaternion holding rads/s:

$$qw = (w + xi + yj + zk)(\omega_x i + \omega_y j + \omega_z k)$$

Collected terms for (w,i,j,k)

$$-x\omega_x - y\omega_y - z\omega_z = (-x\omega_x - y\omega_y - z\omega_z)1$$

$$(w\omega_x i + y\omega_z i - z\omega_y k i) = (w\omega_x + y\omega_z - z\omega_y)i$$

$$(w\omega_y j - x\omega_z j + z\omega_x k j) = (w\omega_y - x\omega_z + z\omega_x)j$$

$$w\omega_z k + x\omega_y k - y\omega_x k = (w\omega_z + x\omega_y - y\omega_x)k$$

$$qw = (-x\omega_x - y\omega_y - z\omega_z) + (w\omega_x + y\omega_z - z\omega_y)i + (w\omega_y - x\omega_z + z\omega_x)j + (w\omega_z + x\omega_y - y\omega_x)k$$

From these equations each component of the quaternion can be defined:

$$q_w = (-x\omega_x - y\omega_y - z\omega_z)1$$

$$q_x = (w\omega_x + y\omega_z - z\omega_y)i$$

$$q_y = (w\omega_y - x\omega_z + z\omega_x)j$$

$$q_z = (w\omega_z + x\omega_y - y\omega_x)k$$

Quaternion integration applying to a program:

since: $Q_{new} = Q \cdot q_{step} = Q \left(1 + \frac{dt}{2} (\omega_x i + \omega_y j + \omega_z k) \right) = Q + Q \frac{dt}{2} (\omega_x i + \omega_y j + \omega_z k)$

$$\frac{dQ}{dt} = \frac{1}{2} Q \omega_q = \frac{1}{2} Q (\omega_x i + \omega_y j + \omega_z k)$$

Input angular velocity	Small angle of in dt	Discrete integration
$\frac{dq_w}{dt} = \frac{1}{2} (-x\omega_x - y\omega_y - z\omega_z)1$	$dq_w = \frac{dq_w}{dt} dt$	$q_{w(new)} = q_{wprev} + dq_w$
$\frac{dq_x}{dt} = \frac{1}{2} (w\omega_x + y\omega_z - z\omega_y)i$	$\frac{dq_x}{dt} = \frac{dq_x}{dt} dt$	$q_{x(new)} = q_{xprev} + dq_x$
$\frac{dq_y}{dt} = \frac{1}{2} (w\omega_y - x\omega_z + z\omega_x)j$	$\frac{dq_y}{dt} = \frac{dq_y}{dt} dt$	$q_{y(new)} = q_{yprev} + dq_y$
$\frac{dq_z}{dt} = \frac{1}{2} (z\omega_x j - z\omega_y k i - z\omega_z)j$	$\frac{dq_z}{dt} = \frac{dq_z}{dt} dt$	$q_{z(new)} = q_{zprev} + dq_z$

These are all the operations used to approximate small angle quaternions then to integrate them in a discrete manner, the resolution has to do with the frequency of the program and the gyro resolution. Generally the program is in this form whe written in C:

Quaternion integration in C
<pre>float qw = q[0], qx = q[1], qy = q[2], qz = q[3]; // q_dot = 0.5 * q ⊗ [0, gx, gy, gz] float q_dot_w = 0.5f * (-qx*gx - qy*gy - qz*gz); // scalar part float q_dot_x = 0.5f * (qw*gx + qy*gz - qz*gy); // i part float q_dot_y = 0.5f * (qw*gy - qx*gz + qz*gx); // j part float q_dot_z = 0.5f * (qw*gz + qx*gy - qy*gx); // k part // Euler integration q[0] += q_dot_w * dt; q[1] += q_dot_x * dt; q[2] += q_dot_y * dt; q[3] += q_dot_z * dt;</pre>

This program is the exact most efficient method of applying Quaternions on a programmable controller, the main issue is when using Euler integration there's a small amount of error which builds up after thousands of readings which may take seconds to minimize this there's a normalization step which can be used. This step is rather straight forward since for quaternions ($\sqrt{i^2 + j^2 + k^2} = 1$) so by applying in the program:

<pre>float norm = sqrtf(q[0]*q[0] + q[1]*q[1] + q[2]*q[2] + q[3]*q[3]); q[0] /= norm; q[1] /= norm; q[2] /= norm; q[3] /= norm;</pre>

This takes the |Q| and divides each Pure component of the Quaternion by it, to make more sense out of it say |Q| < 1 then the 4d unit sphere is smaller than it should be for valid rotation so by dividing by |Q| they scale up to the correct size till |Q| = 1 if |Q| > 1 it's the opposite then the sphere is bigger so dividing the Pure components will scale it down to the correct sphere size.

That's all for Quaternions (at least for now)